

מה זה AI?

אין הגדרות מוצלחות ל-AI - רק חלקיות.
בסדר מהמרוחק ביותר לקרוב ביותר:

1 הגדרה תרבותית

בניגוד למערכות קלאסיות של סיבוכיות, על AI מדברים כבר המון שנים. גם הסיפורים על הגולם מפראג ועל פרנקנשטיין הם בעצם AI.

- בד"כ בתרבות המערבית AI זה דבר מפחיד שבא להרוג אותנו ו/או לקחת להחליף אותנו. בתרבויות אחרות זה לא ככה.
- הרובוטים של אסימוב, בהגדרה, לא יכולים לפגוע בבני אדם - אבל עדיין קורים כל מיני דברים.

2 הגדרה טכנית

“מה שהתעשייה רוצה” - הגדרה קצת מוזרה, כי התעשייה היא לא מונעת ממניעים מדעיים.

- בשנות ה-80 וה-90 היה בעיקר Expert Systems או Knowledge-Based Systems. מערכות שמקדדות הרבה מידע.

- הדרך הנפוצה ביותר - rule-based systems.
- כבר בשנות ה-90 היו מערכות ברמה מאוד גבוהה שיכלו לשחק שחמט ולתת דיאגנוזות רפואיות.
- היו שפות יעודיות כמו prolog.
- השפה הלא-יעודית שהכי השתמשו בה לתחום הייתה lisp.
- השתמשו בחומרה יעודית.

- בשנות ה-2000 התחילו לדבר על Agents - סוכנים שעושים דברים בשבילך.

- סירי ואלקסה
- מערכות CRM וworkflows שיודעות לנהל דברים בתוך הארגון.
- מערכות OCR שיודעות לקרוא.
- אופטימיזציות של מסלולים (Waze)
- אופטימיזציות של לוגיסטיקה.
- בעיקר שפות עיליות - Python, C#, Java.
- בשנות ה-2010 יש פריצת דרך ברשתות נייורוניות:

- גם מבחינת אלגוריתמים.
- גם מבחינת חומרה - מבינים שאפשר לעשות גם דברים לא-גרפיים עם כרטיסי מסך.

- האינטרנט איפשר להשיג המון נתונים בשביל האלגוריתמים האלה.
- התפתחות של למידה עמוקה - שדה רחב, שמתחיל לצבור תאוצה בשנים האלה.
- שנות ה-2020 (העידן הנוכחי) - Generative AI:
- מודלים שמאפשרים לייצר פלטים לקלטים שלא היו ב-data.
- בעיקר LLMים. סוכני LLM.

3 הגדרה מדעית

הגדרה פונקציונלית

מה זה אינטליגנציה?

- לפי דאקרט: אינטליגנציה זה יכולת לייצר משפטים חדשים, שלא היו במקור.
 - לפי טורינג: אינטליגנציה זה יכולת לנהל שיחה ברמה אנושית
- שניהם מתמקדים ביכולת הדיבור.
- דקארט, 1637, טען שאי אפשר לבנות אוטומט שיוכל להגיב באופן הולם לכל מה שיאמרו לו.
 - טורינג, 1950, הציע את The Imitation Game - שידוע כיום בתור "מבחן טורינג". יש שלושה משתתפים:
 - מחשב - A
 - בן אדם - B
 - השופט - C
- השופט שואל שאלות, ואם הוא לא מצליח להסיק מהתשובות מי המחשב - הוא עמד במבחן.
- זוהי הגדרה פונקציונלית. מערכות AI הצליחו לעבור את המבחנים האלה כבר משנות ה-60, אבל אנחנו לא מקבלים אותן בתור AI "אמיתית".
- למשל ELIZA - מערכת ש"פשוט" מנסחת כל מה שהאדם אומר לה בתור שאלה, אבל עשתה את זה מספיק טוב שהיא עברה את מבחן טורינג.
 - ב-1994 היו chatbot ב-Multi-User Dungeons ששחקנים אנושיים חשבו שהם בני אדם.
 - ועוד

הגדרה אחרת - "AI זה כל מה שאנחנו לא מבינים במדעי המחשב"

אפשר לחלק את סוגי AI ל:

- Strong AI זה סוכן שלם שידוע לעשות הכל.

• Weak AI יודע לעשות משהו ספציפי - למשל לשחק שחמט או גו - הרבה פעמים הרבה יותר טוב מבן אדם.

מבחן טורינג אמנם מדבר על שיחה - פעולה מסוימת - אבל מכיוון שאפשר לשאול את המחשב על כל דבר, הוא צריך להיות Strong AI. ברגע אנחנו מבינים איך לעשות משהו, אנחנו כבר לא מוקסמים ממנו - ולכן ניתן לתת הגדרה אחרת:

“AI זה כל מה שאנחנו לא מבינים במדעי המחשב”
בתוך מדעי המחשב, כל עוד אנחנו לא מבינים מה הוא עושה אפשר לסווג אותו בתור Weak AI, וברגע שנבין הוא יהפוך להיות תחום אחר.

כל ההגדרות האלו הן פונקציונליות

הן לא עוסקות בשאלה “מה היא אינטליגנציה” או “מה היא קוגניציה”, אלא מניחות מראש שבני אדם הם אינטליגנטיים - ומגדירים אינטליגנציה המלאכותית לפי ההשוואה לביצועים של בני האדם.
זו הגדרה לא טובה, כי כל דבר לפני שמבינים אותו מסווגים כ-AI. פעם גם תוכנות/אוטומטים שמבצעים חישובים מתמטיים נחשבו ל“AI”.

4 הגדרה ישירה

אנחנו נוטים להעריך אינטליגנציה לפי הציפייה שלנו לרמת הידע. לכן:
“אינטליגנציה היא היכולת לפצות על העדר ידע”
הרעיון הזה הוצע ע"י Newell ב-1982. הוא אומר שכל מערכת אפשר לתאר בכמה רמות - למשל מחשב, אפשר לתאר בסדר עולה ע"י:

1. חשמל שעובר מנקודה לנקודה
2. מעגלים אלטרוניים
3. קומבינטוריקה ולוגיקה סדרתית
4. רמת הCPU והאוגרים
5. רמה סימבולית, של תוכניות

כל רמה היא מקרה פרטי של הרמה שמתחתיה.
מעל הרמה הסימבולית, יש את “רמת הידע”. היא מתארת סוכנים. אם בכל רמה אפשר לתאר מה עובר מאיפה לאיפה, אז כשמדובר בסוכנים מה שזורם בתוך המערכת זה הידע, והרכיבים הם אופרטורים שעושים טרנספורמציה על הידע. ניואל טען שמערכת כזאת אמורה לדעת מה המטרה שלה ולפי זה לדעת איזה פעולות לבחור.
זו לא הגדרה מכאנית, אלא תיאור של התנהגות - למה בדיוק מצפים.
אם יש לסוכן ידע (למשל, התשובות לשאלות במבחן) אז הוא יכול להשתמש בו בשביל להגיע למטרה (למשל, ציון גבוה)

הגדרה: אופרטורים הם רכיבים שלוקחים ידע קיים ויוצרים מתוכו ידע חדש.
למשל - חוק הדדוקציה (מודוס פונס)

כל אופרטור כזה לוקח פיסות ידע קיימות - "believes" - ויוצר מתוכן belief חדש. בהינתן זמן אינסופי, אפשר לעבור בלולאה על בסיס הידע הקיים ולהרחיב אותו כל פעם. בסוף נגיע ל"knowledge closure" - כל הידע שאפשר להגיע אליו בהינתן בסיס ידע B .

הגדרה: סוכן אידאלי הוא סוכן שיש לו את knowledge closure.

- או שהוא יודע כבר את התשובה, ואז יכול לענות עליה.
- או שהתשובה לא נמצאת בknowledge closure - ומכאן ש"אני לא יודע" זו התשובה הנכונה.

בפועל הידע תמיד חלקי, ולכן צריך אינטליגנציה - תהליך חישובי שמפצה על היעדר ידע. זהו תהליך חישובי, ולכן ניתן לדבר עליו בכל המונחים הרגילים של מדעי המחשב (סיבוכיות וכו').

מה יהיה בקורס

יש שלושה סוגים של Inference:

- דדוקציה: בהינתן x ופונקציה $f()$ למצוא y המקיים $y = f(x)$. לא דווקא מספרים.
- אבדוקציה: היכולת לעשות הסקה מהתוצאה לסיבה. כלומר בהינתן y ו $f()$, למצוא x המקיים $y = f(x)$.
- בגלל שיש הרבה אפשרויות, זה הרבה פעמים לא נכון לוגית. יש הרבה דברים שיכולים להסביר את אותו הדבר.
- אינדוקציה: בהינתן x ו y , למצוא $f()$ המקיים $f(x) = y$.
- גם כאן יש הרבה מודלים אפשריים.

הקורסה הזו מלמד איך לבנות מערכות שפותרות בעיות - בניגוד לקורס למידת מכונה שמלמד איך לבנות מערכות שלומדות דברים. GenAI נבנה ע"י כך שמלמדים מערכות איך להשלים תבניות. בהינתן $Part(x)$ ו x , למצוא $f()$ המקיים $f(Part(x)) = x$. זו מערכת שעושה אפרוקסימציה לידע חסר. נתעסק בשני סוגי בעיות:

1. החלטה בודדת (נקודה במרחב)

2. רצף של החלטות (נתיב במרחב)

לא ניגע ב:

1. מרחב רציף (שלא ניתן לייצג בצורה בדידה)
2. פעולות פיזיות. נתעסק רק בפעולות בעולם הווירטואלי. זה אומר, למשל, שלא תמיד אפשר להחזיר פעולות לאחור - לא בלי לשלם על זה מחיר.
3. אינטראקטיביות - עירוב בני אדם בתהליך פתירת הבעיה.

בעיות חיפוש

די הרבה בעיות דטרמיניסטיות אפשר להציג ע"י בעיות חיפוש. הבעיה היא שהגרף הוא לא תמיד גרף נחמד. כדי לייצג גרף, צריך:

- קדקדים
- קשתות
- מאיפה מתחילים
- מטרות אופטימיזציה:
 - לאיפה מגיעים?
 - רוצים מסלול קצר או ארוך?

הגרף הוא לא בהכרח מפה של מקומות. למשל הוא יכול להיות מפה של מצבים - איזה פעולות מביאות מאיזה מצב לאיזה מצב.

הגדרה: "תכנית" - אוסף של מהלכים שמביא ממצב התחלטי למצב מטרות.

מחפשים פתרונות עם תכונות מסוימות (למשל זמן קצוב)
נשים לב: למצוא "פתרון כלשהו" ולמצוא "פתרון אופטימלי" זה בעיות שונות.
יש 3 דרכים לפתור:

1. "תכונות" - הגדרה ידנית של תכנית הפתרון

2. פתרון מבוסס מודל

3. למידה

נתמקד בפתרונות מבוססי מודל.
תכונות הבעיה:

- מרחב מצבים סופי S . נניח לאורך הקורס שיש מספר סופי של מצבים.
- מצב התחלתי $s_0 \in S$.
- אוסף יעדים $S_G \subseteq S$.
- פעולות אפשריות (applicable actions) $A(s) \subseteq A$ לכל $s \in S$
- פונקציית מעבר (transition) $s' = f(a, s)$ לכל $a \in A(s)$
- פונקציית מחיר $c : A \rightarrow [0, \infty)$

פתרון הוא רצף של פעולות אפשריות שממפות את s_0 לתוך S_G . פתרון אופטימלי ממזער את c .
הגדרת הבעיה נותנת לנו גרף אבסטרקטי - שבדרך כלל יהיה גדול מכדי להכניס לזיכרון של מחשב.

אנחנו נמצאים בעולם דטרמיניסטי - מכל קודקוד יכולה לצאת רק קשת אחת **עבור כל פעולה**. פעולה על מצב מסויים לא יכולה להביא ליותר ממצב אחד - אחרת אנחנו לא במערכת דטרמיניסטית.

אם הגרף קטן, אפשר להריץ דייקסטרה - וזה גם נותן את המסלול האופטימלי. אבל כמות המצבים האפשריים יכולה להיות מאוד מאוד גדולה - יותר מכמות האטומים ביקום. אבל אנחנו לא תמיד צריכים את כל המצבים כדי לפתור את הבעיה. אם יש לנו יוריסטיקה טובה, אנחנו יכולים לבנות עץ חיפוש. הגדרה: עץ החיפוש הוא תת-סביבה קטנה של מערכת המצבים שלנו.

אנחנו נפתח את עץ המצבים עד שנגיע למצב המטרה.

נשים לב: לא את כל הבעיות אפשר לפתור ככה.

תכונות הבעיות (לפחות לגבי ההרצאות הקרובות)

- דטרמיניסטיות: בהינתן פעולה וקודקוד, נוכל לתת בהסתברות 1 לאן הפעולה תיקח אותנו - או להגיד שאי אפשר להפעיל את הפעולה על הקודקוד.

— אם אין דטרמיניסטיות, הפעולות הופכות לסטוכסטיות (הסתברויות)

- אובסרביליות: יש לנו ידע מלא על המצבים.

סוגי מטרות

- הגעה למצב מסויים
- להישאר לנצח בטווח מסויים
- למקסם הסתברות להגיע למצב מסויים
- למקסם את תוחלת התגמול או למזער את תוחלת המחיר

הגדרה

מערכת מעברים דטרמיניסטית היא:

$$\langle S, S_0, A, S_G, c \rangle$$

כאשר:

S מרחב המצבים הסופי

$s_0 \in S$ מצב התחלתי

$S_G \subseteq S$ אוסף מצבי יעד

$A(s) \subseteq A$ פעולות אפשריות לכל $s \in S$

$s' = f(a, s)$ פונקציית מעבר

$c : A \rightarrow [0, \infty)$ פונקציית מחיר

דוגמה: עולם הבלוקים (Block world)

- חוקים: מיקום על השולחן לא משנה
 - מיקום על בלוק לא משנה
 - בלוק יכול להיות מונח ישירות על בלוק אחד אחר בלבד
 - על בלוק יכול להיות מונח ישירות רק בלוק אחד
- מספר המצבים האפשריים גדל אקפוננציאלית במספר הקוביות.

בעיות תכנון

נצרך למצוא פתרונות בעולם מסובך.

Progression search

מתחילים ממצב התחלתי, ומתקדמים צעד צעד.

מימוש - קדקדי חיפוש ומצבי חיפוש

- מצבים s - מערכת המצבים במודל המתמטי
 - קודקדים σ - מייצרים אותם במהלך החיפוש. כל קדקד מייצג מצב, אבל יכולים להיות כמה מצבים שמייצגים אותו מצב.
- כל קודקוד מחזיק את המצב, את ההורה, את הפעולה שהביאה אליו, ואת העלות להגיע עד אליו.
- קודקוד בעצם מייצג מסלול. התקדמות בחיפוש.
- מצבי החיפוש וקודקודי החיפוש מייצרים גרפים שונים, אבל אפשר למפות מסלולים מאחד לאחר. קודקודי החיפוש מייצרים עץ חיפוש.
- אלגוריתם חיפוש צריך את הפונציות:
- $init()$ - מייצר את המצב ההתחלתי.
 - $is - goal(s)$ - בודק אם מצב מסוים הוא מצב מטרה
 - $succ(s)$ - מייצר את successor states של מצב s , יחד עם הפעולות הנדרשות כדי להגיע לכל מצב בן.

סוגי חיפוש

- Uninformed search
- Heuristic search